



Las variables

¿Qué es una variable?

Para el compilador una variable es una entidad que tiene:

- Un **nombre**
- Un **tipo de dato**
- Un **alcance**

En ejecución una variable tiene un **valor** almacenado en **memoria**

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

Las variables en Java

En Java el **tipo** de una variable puede ser:

- Elemental
- Una clase

El **valor** de una **variable** de un **tipo elemental** pertenece al conjunto de valores definido por el tipo. En Java todos los tipos elementales están provistos por el lenguaje.

El **valor** de una **variable** de un **tipo clase** es una **referencia**. Una referencia puede ser **indefinida**, **nula** o estar **ligada** a un objeto.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

Las variables en Java

En Java el programador puede declarar variables para definir

- Atributos de clase o de instancia
- Parámetros
- Variables locales

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

El alcance

¿Qué es el alcance de una variable?

El alcance de una variable es el segmento de código dentro del cual la variable :

- es visible
- puede ser usada

Una variable puede mantener un valor en memoria pero no ser visible

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

El ambiente de referenciamiento

¿Qué es el ambiente de referenciamiento?

El ambiente de referenciamiento de una instrucción es el conjunto de identificadores que pueden ser usados o referenciados en esa instrucción.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

Los mensajes

En la POO cuando un objeto recibe un mensaje ejecuta uno de los métodos definidos por su clase.
La ejecución de un método puede modificar el estado interno del objeto que recibe el mensaje.
La ejecución de un mensaje también puede retornar un resultado.

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

Los parámetros

Un mensaje puede incluir uno o más **argumentos** o **parámetros reales**.

El número y tipo de los parámetros reales tiene que coincidir con los **parámetros formales** que establece el método.

En Java el **pasaje de parámetros es por valor**, en el momento que se inicia la ejecución de un servicio, se reserva espacio en memoria para los parámetros formales y se inicializan con los valores de los argumentos.

Cuando el método termina de ejecutarse el espacio reservado para los parámetros formales se libera.

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

La clase Punto

Punto <<atributos de instancia>> x,y:real <<Constructores>> Punto(x,y:real) <<Comandos>> establecerX(x:real) establecerY(y:real) copy (p:Punto) <<Consultas>> obtenerX():real obtenerY():real equals(p:Punto):boolean distancia(p:Punto):real clone():Punto toString():String	copy (p:Punto) Requiere p ligado equals(p:Punto):boolean Requiere p ligado distancia(p:Punto):real Requiere p ligado. La distancia entre dos puntos se calcula aplicando Pitágoras toString():String Retorna las coordenadas entre paréntesis, es decir con formato "(x,y)"
---	--

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

La clase Punto

La clase **Punto** puede ser construida y verificada sin necesidad de saber para qué va a ser usada.

Las clases que usan a **Punto** como un tipo de dato pueden declarar variables, crear objetos, enviarles mensajes, sin conocer cómo es la representación de los valores ni la implementación de las operaciones.

La abscisa y la ordenada de un objeto de clase **Punto** solo pueden modificarse usando los servicios provistos por la clase.

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

La clase Tester

```
Punto p1,p2,p3,p4,p5;
p1 = new Punto (2,0);
p2 = new Punto (2,0);
p3 = p1;
p4 = new Punto (3,-1);
p5 = new Punto (3,1);
p5.copy(p1);
boolean b1 = p1 == p2;
boolean b2 = p1 == p3;
boolean b3 = p1 == p4;
boolean b4 = p1 == p5;
```

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

El operador relacional == compara los valores de las variables. En el caso de variables de tipo clase, se comparan **referencias**.

La clase Tester

```
Punto p1,p2,p3,p4,p5;
p1 = new Punto (2,0);
p2 = new Punto (2,0);
p3 = p1;
p4 = new Punto (3,-1);
p5 = new Punto (3,1);
p5.copy(p1);
boolean b1 = p1.equals(p2);
boolean b2 = p1.equals(p3);
boolean b3 = p1.equals(p4);
boolean b4 = p1.equals(p5);
```

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

El método **equals** compara el estado interno de los objetos.

La clase Tester

```
Punto p1,p2,p3,p4;
p1 = new Punto (2,0);

boolean b = p1.equals(p2);

p3.establisherX(-1);
p4.copy(p1);
p1.copy(p4);
```



01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

La clase Tester

```
Punto p1,p2;
p1 = new Punto (2,0);
p2 = null;
boolean b = p1.equals(p2);
```



01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

La clase Recta

```
Punto
<<atributos de instancia>>
x,y:real

<<Constructores>>
Punto(x,y:real)
<<Comandos>>
establecerX(x:real)
establecerY(y:real)
copy (p:Punto)
<<Consultas>>
obtenerX():real
obtenerY():real
equals(p:Punto):boolean
distancia(p:Punto):real
clone():Punto
toString():String
```

```
Recta
<<atributos de instancia>>
p1: Punto
p2: Punto

<<Constructores>>
Recta(p1,p2: Punto)
<<Comandos>>
copy(l:Recta)
<<Consultas>>
obtenerP1():Punto
obtenerP2():Punto
equals(l:Recta):boolean
clone():Recta
longitud():real
mayor(l: Recta):boolean
toString():String
```

Asociación

Dependencia

Dependencia

No necesitamos conocer la implementación de Punto para implementar la clase Recta, solo necesitamos conocer el diseño de la clase

01100
10011
10110
01110
10
10110
10110
1001
0 11
0 0
1

La clase Recta

```
class Recta{
//Atributos de instancia
private Punto p1,p2;
//Constructor
public Recta(Punto p1, Punto p2){
this.p1=p1;
this.p2=p2;
}
```

```
Recta
<<atributos de instancia>>
p1: Punto
p2: Punto

<<Constructores>>
Recta(p1,p2: Punto)
<<Comandos>>
copy(l:Recta)
<<Consultas>>
obtenerP1():Punto
obtenerP2():Punto
equals(l:Recta):boolean
clone():Recta
longitud():real
mayor(l: Recta):boolean
toString():String
```

01100
10011
10110
01110
10110
10110
10110
1001
0 11
0 0
1

La clase Recta

```
//Consultas
public Punto obtenerP1() {
return p1;
}

public Punto obtenerP2() {
return p2;
}
```

```
Recta
<<atributos de instancia>>
p1: Punto
p2: Punto

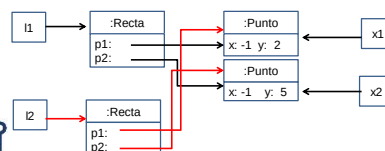
<<Constructores>>
Recta(p1,p2: Punto)
<<Comandos>>
copy(l:Recta)
<<Consultas>>
obtenerP1():Punto
obtenerP2():Punto
equals(l:Recta):boolean
clone():Recta
longitud():real
mayor(l: Recta):boolean
toString():String
```

01100
10011
10110
01110
10110
10110
10110
1001
0 11
0 0
1

El diagrama de objetos

```
class Tester{
public static void main (String arg[]){
Punto x1,x2;
Recta l1,l2;
x1 = new Punto (-1,2); x2 = new Punto (-1,5);
l1 = new Recta (x1,x2);
l2 = l1.clone();
}
```

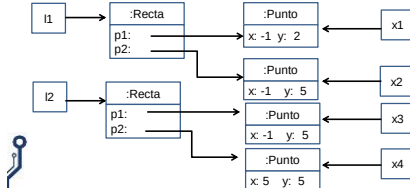
CLONE SUPERFICIAL



01100
10011
10110
01110
10110
10110
1001
0 11
0 0
1

El diagrama de objetos

```
class Tester{
public static void main (String arg[]){
    Punto x1,x2,x3,x4;
    Recta l1,l2;
    x1=new Punto(-1,2); x2=new Punto(-1,5);
    x3 = x2.clone(); x4=new Punto(5,5);
    l1=new Recta (x1,x2); l2=new Recta(x3,x4);
}
```

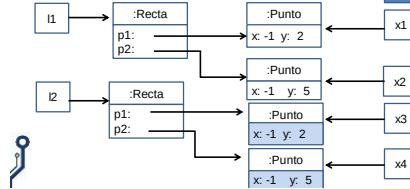


```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

El diagrama de objetos

```
class Tester{
public static void main (String arg[]){
    Punto x1,x2,x3,x4;
    Recta l1,l2;
    x1=new Punto(-1,2); x2=new Punto(-1,5);
    x3 = x2.clone(); x4=new Punto(5,5);
    l1=new Recta (x1,x2); l2=new Recta(x3,x4);
    l2.copy(l1);
}
```

COPY EN PROFUNDIDAD



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

La clase Recta

```
Recta(p1,p2: Punto)
Si el parámetro p1 no está ligado, inicializa
p1 con (0,0). Si el parámetro p2 no está
ligado, inicializa p2 con (0,1)

copy(l: Recta)
Requiere l ligado, se implementa en
profundidad

equals(l: Recta): boolean
Requiere l ligado, se implementa superficial

clone(): Recta
Se implementa superficial

mayor(l: Recta): boolean
Requiere l ligado, retorna verdadero si la
longitud de la recta que recibe el mensaje es
mayor que la longitud de l
```

```
Recta
<<atributos de instancia>>
p1: Punto
p2: Punto

<<Constructores>>
Recta(p1,p2: Punto)

<<Comandos>>
copy(l: Recta)

<<Consultas>>
obtenerP1(): Punto
obtenerP2(): Punto
equals(l: Recta): boolean
clone(): Recta
longitud(): real
mayor(l: Recta): boolean
toString(): String
```

De acuerdo a este diseño p1 y p2 SIEMPRE estarán ligados

La clase Recta

```
public void copy(Recta l){
    /*Requiere l ligado, se implementa
    en profundidad*/
    p1.copy(l.obtenerP1());
    p2.copy(l.obtenerP2());
}
```

```
Recta
<<atributos de instancia>>
p1: Punto
p2: Punto

<<Constructores>>
Recta(p1,p2: Punto)

<<Comandos>>
copy(l: Recta)

<<Consultas>>
obtenerP1(): Punto
obtenerP2(): Punto
equals(l: Recta): boolean
clone(): Recta
longitud(): real
mayor(l: Recta): boolean
toString(): String
```

La clase Recta

```
public boolean equals(Recta l){
    /*Requiere l ligado, se implementa
    superficial*/
    return ((p1==l.obtenerP1()) &&
            (p2==l.obtenerP2()));
}

public Recta clone(){
    /*Se implementa superficial
    return new Recta(p1,p2);
}
```

```
Recta
<<atributos de instancia>>
p1: Punto
p2: Punto

<<Constructores>>
Recta(p1,p2: Punto)

<<Comandos>>
copy(l: Recta)

<<Consultas>>
obtenerP1(): Punto
obtenerP2(): Punto
equals(l: Recta): boolean
clone(): Recta
longitud(): real
mayor(l: Recta): boolean
toString(): String
```

La clase Recta

```
public double longitud(){
    return p1.distancia(p2);
}

public boolean mayor(Recta l){
    /*Requiere l ligado, retorna
    verdadero si la longitud de la recta
    que recibe el mensaje es mayor que
    la longitud de l*/
    return longitud() >
           l.longitud();
}

public String toString(){
    return p1.toString()+
           "-" + p2.toString();
}
```

```
Recta
<<atributos de instancia>>
p1: Punto
p2: Punto

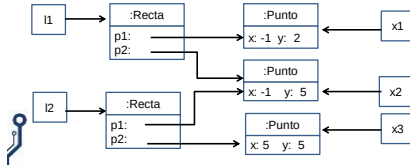
<<Constructores>>
Recta(p1,p2: Punto)

<<Comandos>>
copy(l: Recta)

<<Consultas>>
obtenerP1(): Punto
obtenerP2(): Punto
equals(l: Recta): boolean
clone(): Recta
longitud(): real
mayor(l: Recta): boolean
toString(): String
```

El diagrama de objetos

```
class Tester{
public static void main (String arg[]){
    Punto x1,x2,x3;
    Recta l1,l2;
    x1=new Punto (-1,2); x2=new Punto (-1,5); x3=new Punto(5,5);
    l1=new Recta (x1,x2); l2=new Recta(x2,x3);
}
```

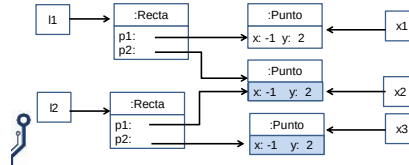


```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

El diagrama de objetos

```
class Tester{
public static void main (String arg[]){
    Punto x1,x2,x3;
    Recta l1,l2;
    x1=new Punto (-1,2); x2=new Punto (-1,5); x3=new Punto(5,5);
    l1=new Recta (x1,x2); l2=new Recta(x2,x3);
    l2.copy(l1);
}
```

COPY EN PROFUNDIDAD



```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

La clase Tester

```
class Tester{
public static void main (String arg[]){
    Punto x1,x2,x3,x4,x5;
    Recta l1,l2,l3,l4,l5;

    x1 = new Punto (-1,2);
    x2 = new Punto (-1,2);
    x3 = x1;
    x4 = x1.clone();
    x5 = new Punto (-1,5);
    l1 = new Recta (x1,x5);
    l2 = new Recta (x2,x5);
    l3 = new Recta (x3,x5);
    l4 = new Recta (x4,x5);
    l5 = l1.clone();
}
```

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

La clase Tester

```
System.out.println(l1.equals(l2));
System.out.println(l1.equals(l3));
System.out.println(l1.equals(l4));
System.out.println(l1.equals(l5));
```

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

La clase Circulo

Punto	Circulo
<<atributos de instancia>> x,y:real	<<atributos de clase>> pi:real <<atributos de instancia>> radio:real centro: Punto
<<Constructores>> Punto(x,y:real) <<Comandos>> establecerX(x:real) establecerY(y:real) copy (p:Punto) <<Consultas>> obtenerX():real obtenerY():real equals(p:Punto):boolean distancia(p:Punto):real clone():Punto toString():String	<<Constructores>> Circulo(r:real, p: Punto) <<Comandos>> trasladar(p:Punto) escalar(r:real) copy (c: Circulo) <<Consultas>> obtenerCentro(): Punto obtenerRadio():real area():real perimetro():real pertenece(p:Punto):boolean equals(c:Circulo):boolean clone():Circulo toString():String

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

Las clases asociadas

Las clases **Recta** y **Punto** están relacionadas por **asociación**.

La clase **Recta** tiene dos atributos de clase **Punto**.

Existe también una relación de **dependencia** entre **Recta** y **Punto**. El constructor de **Recta** recibe dos parámetros de clase **Punto**.

La clase **Recta** usa los servicios provistos por la clase **Punto** conociendo su interfaz, su funcionalidad y sus responsabilidades, no su implementación.

La clase **Punto** se diseña, implementa y verifica sin saber que va a ser usada por la clase **Recta**.

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

Las clases asociadas

Las clases **Circulo** y **Punto** están relacionadas por **asociación**.
La clase **Circulo** **tiene un** atributo de clase **Punto**.
La clase **Circulo** **conoce** la interfaz de **Punto**, su funcionalidad y sus responsabilidades, no su implementación. Es decir, sabiendo **qué** hacen, sin saber **cómo** lo hacen.
La clase **Punto** se diseña, implementa y verifica sin saber que va a ser usada por la clase **Circulo**.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

31

Las clases asociadas

Existe también una relación de dependencia entre **Circulo** y **Punto**.

El constructor de **Circulo** y el método **trasladar** reciben un parámetro de clase **Punto**.

El método **obtenerCentro()** retorna como resultado un objeto de clase **Punto**.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

32

La clase Circulo

```
Circulo(r:real,p:Punto)
Requiere p ligado y r>0

trasladar(p:Punto): centro=p
Requiere p ligado

escalar(r:real):
radio = r Requiere r>0

pertenece(p:Punto)
Retorna verdadero si p pertenece a la
superficie del círculo
Requiere p ligado
```

```
Circulo
<<atributos de clase>>
pi:real
<<atributos de instancia>>
radio:real
centro: Punto
<<Constructores>>
Circulo(r:real, p: Punto)
<<Comandos>>
trasladar(p:Punto)
escalar(r:real)
copy(c: Circulo)
<<Consultas>>
obtenerCentro():Punto
obtenerRadio():real
area():real
perimetro():real
pertenece(p:Punto):boolean
equals(c:Circulo):boolean
clone():Circulo
toString():String
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

La clase Circulo

```
copy(c:Circulo)
Requiere c ligado, se implementa
superficial

equals(c:Circulo):boolean
Requiere c ligado, se implementa en
profundidad

clone():Circulo
Se implementa en profundidad
```

```
Circulo
<<atributos de clase>>
pi:real
<<atributos de instancia>>
radio:real
centro: Punto
<<Constructores>>
Circulo(r:real, p: Punto)
<<Comandos>>
trasladar(p:Punto)
escalar(r:real)
copy(c: Circulo)
<<Consultas>>
obtenerCentro():Punto
obtenerRadio():real
area():real
perimetro():real
pertenece(p:Punto):boolean
equals(c:Circulo):boolean
clone():Circulo
toString():String
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

La clase Circulo

```
public void copy (Circulo c){
//Requiere c ligado, se implementa superficial
radio = c.obtenerRadio();
centro = c.obtenerCentro();
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Copy superficial, asigna la **referencia**.

El centro del círculo que pasó como parámetro pasa a ser también el centro del círculo que recibe el mensaje.

La clase Circulo

```
public boolean equals(Circulo c){
//Requiere c ligado, se implementa en profundidad
return radio == c.obtenerRadio() &&
centro.equals(c.obtenerCentro());
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Igualdad en profundidad, compara el **estado interno** de los objetos asociados.

Compara el estado interno del centro del círculo que pasa como parámetro con el estado interno del centro del círculo que recibe el mensaje.

La clase Circulo

```
public Circulo clone () {
//Se implementa en profundidad
return new Circulo (radio, centro.clone());
}
```

Clone en profundidad, se crea un nuevo objeto de la clase asociada, con el mismo estado interno que el objeto asociado al objeto que recibió el mensaje.

Crea un nuevo círculo con el mismo centro que el círculo que recibió el mensaje.

01100
10011
10110
01110
11100
10011
10110
10110
1001
1 11
0 0
1

Los cambios en el diseño

trasladar(p:Punto): Si p está ligado cambia el centro del círculo, en caso contrario no produce cambios

Cambia el compromiso entre las clases, el cambio es VISIBLE para las clases clientes de Circulo.

```
Circulo
<<atributos de clase>>
pi:real
<<atributos de instancia>>
radio:real
centro: Punto
<<Constructores>>
Circulo(r:real, p: Punto)
<<Comandos>>
trasladar(p:Punto)
escalar(r:real)
copy (c: Circulo)
<<Consultas>>
obtenerCentro():Punto
obtenerRadio():real
area():real
perimetro():real
pertenece(p:Punto):boolean
equals(c:Circulo):boolean
clone():Circulo
toString():String
```

01100
10011
10110
01110
11100
10011
10110
10110
1001
1 11
0 0
1

Los cambios en el diseño

pertenece(p:Punto)
Retorna verdadero si p pertenece a la **circunferencia** del círculo
Requiere p ligado

Cambia la funcionalidad del método pertenece, el cambio es VISIBLE para las clases clientes de Circulo.

```
Circulo
<<atributos de clase>>
pi:real
<<atributos de instancia>>
radio:real
centro: Punto
<<Constructores>>
Circulo(r:real, p: Punto)
<<Comandos>>
trasladar(p:Punto)
escalar(r:real)
copy (c: Circulo)
<<Consultas>>
obtenerCentro():Punto
obtenerRadio():real
area():real
perimetro():real
pertenece(p:Punto):boolean
equals(c:Circulo):boolean
clone():Circulo
toString():String
```

01100
10011
10110
01110
11100
10011
10110
10110
1001
1 11
0 0
1

Los cambios en el diseño

```
class Figura{
...
public void test () {
Punto p1,p2;
Circulo c;
p1=new Punto (1,2) ;
p2=new Punto (-1,2) ;
c=new Circulo (5,p1) ;
if (c.pertenece (p2))
System.out.println ("p2 pertenece a c");
}
```

La clase **Figura** necesita conocer **qué** hace el método **pertenece**, no necesita saber **cómo** lo hace. Si cambia la funcionalidad de **pertenece**, el cambio impacta en la clase cliente.

01100
10011
10110
01110
11100
10011
10110
10110
1001
1 11
0 0
1